03 MAR 2026

AADK TASK 4

Capstone: Full Scrollable List Feature & Documentation

MindMatrix

1. Full Feature Specification

Final Theme: Travel Destinations Explorer
IThis capstone integrates all previous tasks into one complete feature plan.

Feature Description:

This feature displays a scrollable list of travel destinations with images and details. Users can:

- Browse destinations in a scrollable list
- View images of each place
- Mark destinations as Favourite
- Expand an item to read full description
- View responsive layout on phone and tablet

Special Features:

- Responsive layout (Single column on phone, grid on tablet)
- Image loading with Coil (with placeholder and error handling)
- Stateful Favourite toggle
- Expandable list items
- Smooth animations
- Lazy loading for performance

The goal of this feature is to provide a smooth, modern UI experience while maintaining good performance.

2. Comprehensive Data & UI Documentation

A. Data Model

```
data class TravelDestination(
    val id: Int,
    val title: String,
    val country: String,
    val description: String,
    val imageUrl: String,
    val isFavourite: Boolean
)
```

Dataset:

- 10 travel destinations
- Each contains:
  - Unique ID
  - Title
  - Country
  - Description
  - Image URL
  - Favourite flag

State will be managed inside a ViewModel to prevent data loss.

B. Item Interaction Design

Favourite Feature:

- Heart icon at top-right
- Gray when not favourite
- Red when favourite
- Clicking toggles state

State Management:

- Managed inside ViewModel
- List updated using copy()
- LazyColumn uses unique key = item.id

Item Expansion Feature:

- Clicking card expands item
- Full description appears
- Smooth animation using animateContentSize()
- Expansion state stored using rememberSaveable or ViewModel

Collapsed View:

- Short description (2 lines)

Expanded View:

- Full description
- Larger image

C. Layout Variants

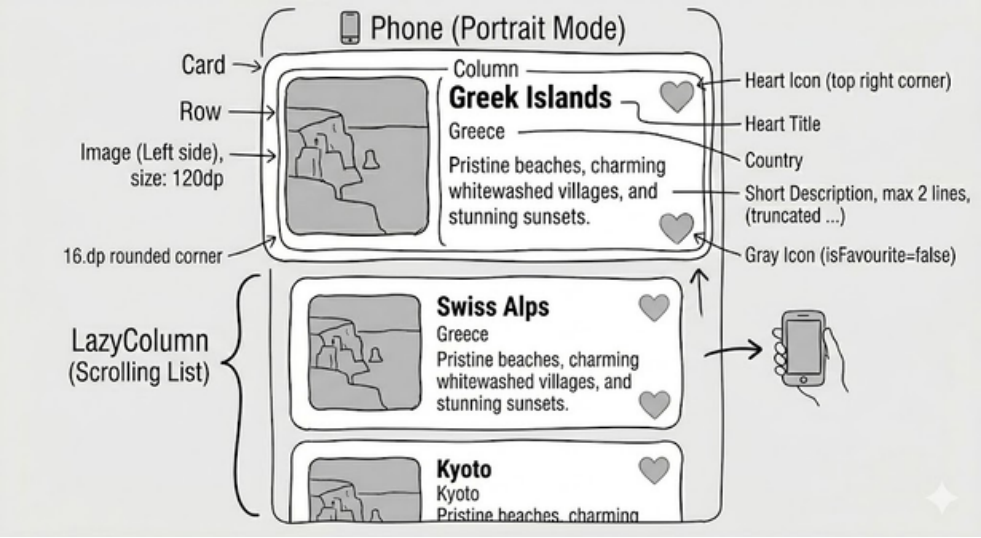
Phone

Component Used:

- LazyColumn

Layout Structure:

- Image size: 120dp
- Card padding: 8dp
- Rounded corners: 16dp

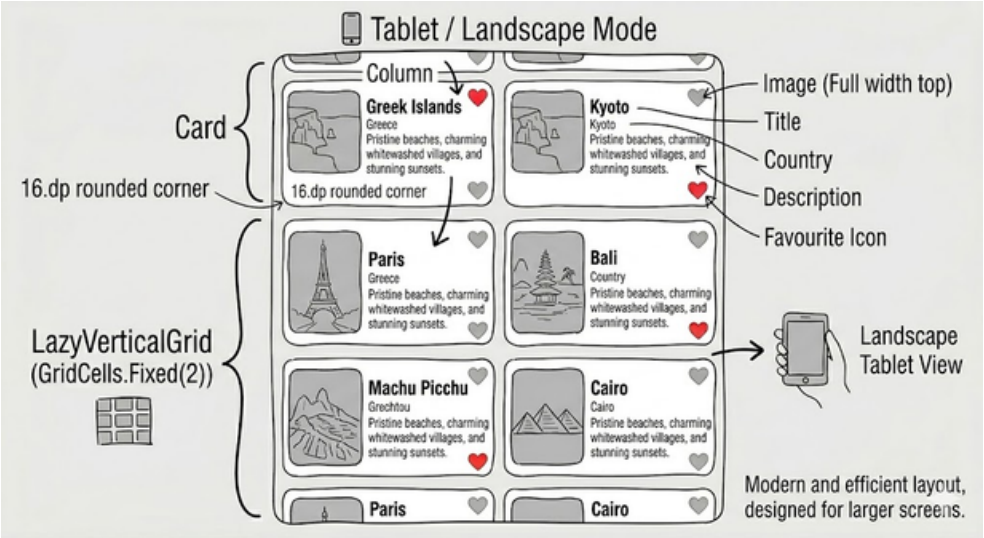


Tablet

Component Used:

- LazyVerticalGrid (GridCells.Fixed(2))

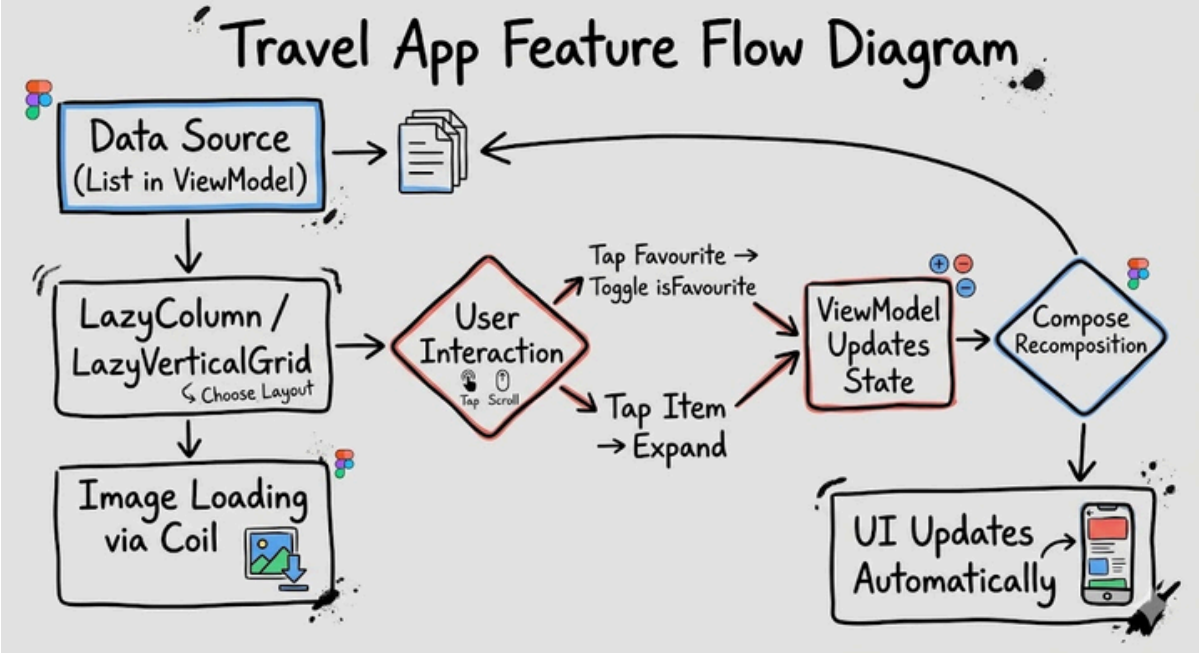
Layout Structure:



This layout uses available space efficiently and looks modern on larger screens.

3. Flow Diagram & Process Map

Below is the overall feature flow:



Where things happen:

- Data stored: ViewModel
- Image loading: Inside Composable using AsyncImage (Coil)
- State update: ViewModel
- UI Feedback: Immediate recomposition

4. Implementation Stack & Timeline Plan

Tools & Dependencies:

- Jetpack Compose
- LazyColumn / LazyVerticalGrid
- ViewModel
- Coil (for image loading)
- Compose Animations
- Material 3 components

Step-by-Step Timeline Plan

Step 1: Basic List UI (Day 1)

- Create data class
- Create dummy dataset
- Display list using LazyColumn

Step 2: Image Loading (Day 2)

- Add Coil dependency
- Load images from URLs
- Add placeholder and error image

Step 3: Interactivity & State (Day 3)

- Implement Favourite toggle
- Connect ViewModel
- Add expansion feature
- Add animation

Step 4: Testing & Optimization (Day 4)

- Test scroll performance
- Test rapid clicking
- Test orientation change
- Test on tablet layout

5. Reflection

This feature became complex because it includes multiple moving parts like responsive layout, image loading, animation, and state management. The biggest challenge was managing state correctly inside Lazy lists without losing data during scrolling or rotation. The main trade-off was between simplicity and rich features — adding expansion and favourites increases complexity but improves user experience.